



HQL and criteria queries

1. What is HQL
2. HQL examples
3. Named queries
4. Criteria queries

1. What is HQL

- Hibernate Query Language (HQL) is an object-oriented query language, similar to SQL.
- HQL works with persistent objects and their properties instead of tables and columns.
- HQL's syntax is very similar to standard SQL.
- Commonly supported clauses in HQL
 - HQL From
 - HQL Join
 - Aggregate Functions
 - Expressions
 - order by and group by
 - subqueries

- HQL is a simple string which will look like SQL query where the name of attributes and classes are referred.
- Simple example for retrieving all employees :

```
Query q = session.createQuery("from Emp",Emp.class);  
List<Emp> emps = q.getResultList();  
for(Emp e : emps) {  
    System.out.println(e);  
} //List - element - Emp object
```
- Example of HQL for getting only few attributes :

```
Query q = session.createQuery("select empid,ename from Emp  
where salary > 20000",Object[].class);  
List<Object []> emps = q.getResultList();  
// List - element - array(record) of object(column)  
for( Object [] e : emps )  
    System.out.println(e[0]+" : "+e[1]);
```

- Hibernate provides Named Query that we can define at a central location and use them anywhere in the code.
- Named queries can be created for both HQL and Native SQL.
- Hibernate Named Query can be defined in Hibernate mapping files or through the use of JPA annotations `@NamedQuery` and `@NamedNativeQuery`.

- Writing named query

```
@Entity
```

```
@Table(name="emp")
```

```
@NamedQuery(name = "getByDept",query="select empid, salary  
from Emp where dept = :dept")
```

```
public class Emp { // }
```

- Using named query

```
Query q = session.createNamedQuery("getByDept",Object[].class);
```

- Criteria queries are a programmatic, type-safe way to express a query.
- These queries use interfaces and classes to represent various structural parts of a query such as the query itself, the select clause, or an order-by, etc.
- CriteriaBuilder interface is the first object that needs to be created for criteria queries.
- Steps to be followed
 1. create criteriabuilder instance
`CriteriaBuilder cb = session.getCriteriaBuilder();`
 2. create criteriaquery instance
`CriteriaQuery<Emp> cquery = cb.createQuery(Emp.class);`
 3. specify the root(table from which the records)
`Root<Emp> root = cquery.from(Emp.class);`

4. specify the properties to be selected
`cquery.select(root);` .
5. prepare the query from the criteria
`Query q = session.createQuery(cquery);`
6. get the result(list) from the query
`List<Emp> emps = q.getResultList();`
`for(Emp e : emps)`
`System.out.println(e.getEmpid()+" : "+e.getEname());`